

□□□ □□□□□□ □□□□□

□□ □□□□□□ Delta

Delta Synchronization

□□□□□ □□□□□ □□□□□□ □□ □□□□□□□□ □□□□□□□□
□□□□□□□□

□□□□□□□□ □□□□□□□□ □□□□□□□□ □□□□□□□

Table of Contents

1.	Introduction to Delta	3
2.	Delta-State CRDT	4
3.	Differential Synchronization	5
4.	Operational Transformation	7
4.1	Delta-State CRDT	7
4.2	Differential Synchronization	8
4.3	Operational Transformation	9
5.	Delta-Operational Transformation	10
6.	Delta-Operational Transformation	11
7.	Delta-Operational Transformation	13
8.	Delta-Operational Transformation	15
9.	Delta-Operational Transformation	17
10.	Delta-Operational Transformation	18
	Delta-Operational Transformation	19

1. Delta

Delta 是 Google 开发的一种用于在移动设备上实现离线优先（Offline-First）架构的数据库。它允许应用在无网络连接的情况下，仍然可以读写数据，并在网络恢复后将本地数据同步到云端。Delta 支持多种数据库引擎，包括 Google Docs、Firebase、CouchDB 等。它通过 Delta 接口与后端服务进行交互，确保数据的一致性和完整性。

1.1 Delta

Delta 是一种用于在移动设备上实现离线优先（Offline-First）架构的数据库。它允许应用在无网络连接的情况下，仍然可以读写数据，并在网络恢复后将本地数据同步到云端。Delta 支持多种数据库引擎，包括 Google Docs、Firebase、CouchDB 等。它通过 Delta 接口与后端服务进行交互，确保数据的一致性和完整性。

Delta 的架构设计旨在简化开发者的工作，使其能够轻松地在移动应用中实现离线功能。它通过 Delta 接口与后端服务进行交互，确保数据的一致性和完整性。

Delta 的架构设计旨在简化开发者的工作，使其能够轻松地在移动应用中实现离线功能。它通过 Delta 接口与后端服务进行交互，确保数据的一致性和完整性。

2. 数据库复制与数据同步

数据库复制是指将主数据库的数据复制到从数据库的过程。复制可以分为全量复制和增量复制。全量复制是指将主数据库中的所有数据复制到从数据库。增量复制是指将主数据库中的增量数据复制到从数据库。增量复制通常使用日志复制（Log Shipping）或基于时间戳的复制（Timestamp-based Replication）来实现。日志复制是指将主数据库的事务日志复制到从数据库，然后从数据库根据日志中的记录来更新数据。基于时间戳的复制是指将主数据库中的数据按照时间戳的顺序复制到从数据库，然后从数据库根据时间戳来更新数据。

2.1 数据库复制与数据同步

数据库复制与数据同步是指将主数据库的数据复制到从数据库，并确保从数据库中的数据与主数据库中的数据保持一致的过程。复制可以分为全量复制和增量复制。全量复制是指将主数据库中的所有数据复制到从数据库。增量复制是指将主数据库中的增量数据复制到从数据库。增量复制通常使用日志复制（Log Shipping）或基于时间戳的复制（Timestamp-based Replication）来实现。日志复制是指将主数据库的事务日志复制到从数据库，然后从数据库根据日志中的记录来更新数据。基于时间戳的复制是指将主数据库中的数据按照时间戳的顺序复制到从数据库，然后从数据库根据时间戳来更新数据。数据库复制与数据同步可以用于提高数据库的可用性和可靠性。当主数据库发生故障时，可以从数据库接管工作，确保数据的连续性和一致性。此外，数据库复制还可以用于数据备份和灾难恢复。通过将数据复制到从数据库，可以在主数据库发生故障时快速恢复数据。数据库复制还可以用于数据分析和报表生成。通过将数据复制到从数据库，可以在不影响主数据库性能的情况下进行数据分析和报表生成。

2.2 数据库复制与数据同步

数据库复制与数据同步可以分为以下几种类型：全量复制（Full Sync）、增量复制（Incremental Sync）、基于时间戳的复制（Timestamp-based Replication）和分布式锁（Distributed Locking）。全量复制是指将主数据库中的所有数据复制到从数据库。增量复制是指将主数据库中的增量数据复制到从数据库。基于时间戳的复制是指将主数据库中的数据按照时间戳的顺序复制到从数据库，然后从数据库根据时间戳来更新数据。分布式锁是指将数据库中的锁分布在多个节点上，以确保数据的一致性和完整性。

□□□□ □□□□ □□□□□□□□ □□□□□□□□. □□□ □□
□□□□□□□□ □□□□□□□□. □□□□□□□□ □□□□□□
□□□□□□□□□□ □□□□ □□□ □□□□□ □□□□□□□□
□□□□□□. □□□□□□□□ □□□□□□□ □□□□□ □□□□□ □□ □□□□□
□□□□□□□□□ □□□□□□□□□ □□□ □□□ □□□□□□□□□ □□□□
□□□□. □□□□□□□□ □□□□□□ □□□□□ □□□□□□□□ □□
□□□□□□ □□□□□□□ □□□□□□□ □□□ □□□□□□□. □□□
□□□□□□□□□ □□□□□□□ □□□□□ □□□□□□ □□□□ □□□□□□
□□□ □□□□□□ **Delta**□□□□ □□□□ □□□ □□□□□ □□□ □□□□□□
□□□□□□□ □□□□□□.

3. 如何選擇合適的 Delta

০০০০০ ০০০০০ ০০০০০০ **Delta** ০০০ ০০০০ ০০০০০ ০০০: ০০০০
 ০০০০০০০০০০ ০০০০ ০০০০০০০০০ ০০০. ০০০০০ ০০ ০০০০০
 ০০০০০০০০০ ০০০০০০০০ ০০ ০০ ০০০০০ ০০০০০০০০ ০০০০
 ০০০০০০০ ০০০০০০ ০০০০০০০০০০০০ ০০০০ ০০০০ ০০০ ০০০০০০০০০০
 ০০০০০০০০ ০০ ০০০০০ ০০০ ০০০০০০০০০০০ ০০০০ ০০০০ ০০০০০০০০
 ০০ ০০০০০০০০০ ০০০০০০০০. ০০০০ ০০০০০০০ ০০০০০ ০০০০০ ০০০০০ ০০
 ০০০০ ০০০০০০০০০০০ ০০০০০০০০০০০ ০০০০০০ ০০০০০০০০ ০০০০০০০

3.1 数据输入与输出

The **Delta** component is responsible for tracking and managing changes in the data. It maintains a **Change Tracker** to record all modifications made to the data. These changes are then propagated to the **Record-level** and **Field-level** components. The **Record-level** component handles changes at the record level, while the **Field-level** component handles changes at the field level. The **Delta Store** is used to store the changes, and the **Outbox** is used to manage the **Sync Queue**. The **Sync Engine** is responsible for synchronizing the data, and the **Conflict Resolver** is used to resolve conflicts. The **Communication Layer** is used to communicate with the external systems.

3.2 内部リンク

□□□□ □□□□□□ □□□□□□□ □□□□□□□□□□ □□□ □□□ □□□□□□
□□□□□ □□□ □□□□□ □□ □□□□□ □□□□□□□□□ □□ □□□□□□□□□.

G-Counter	□□□□ □□□□ □□□	Max()	□□□□□□□□ □□□□ □□□□ □
PN-Counter	□□□□ □□□□ □□□□□□ □□□□□□□□	Max() + Max()	□□□□□□ □□□□□
G-Set	□□□□□□ □□□□□ □□□	Union	□□□□□□ □□□□□
OR-Set	□□□□□□ □□ □□□□□ □□□□ □□	Union - Remove wins	□□□□□□ □□□□□□
LWW-Register	□□□□ □□ □□□ □□□□□	Timestamp-based	□□□□□□□□ □□□□□□

□□□□ 1: □□□□□ CRDT □□□□□□□□ □□□□□□□□□□□□

4.2 Differential Synchronization

□□□□ Neil Fraser □□ Google □□□□□□□□ Differential Synchronization
□□□ 2009 □□□□□ □□ Operational Transformation. □□□□□□□ □□□
□□□□□□□□□□ □□ Google Docs □□□□□□□□ □□□□□□□□ □□
□□□□□ □□□□□□□□. □□□□□ □□□□□□□□□□□ □□□ □□□□□ Shadow
(□□□□)□ □□□ □□□□ □□□□□ □□ □□□□□□□□ □□□□ □□□□□□
□□□□□□ □□□□□ □□□ □□□□□□ □□□□□□□□. □□□□ □□□□□
□□□□□□□□ □□□□□□ □□□□□ □□□□□□ □□□ □□□□□□
□□□□□□□ □□□□ Shadow □□□□□ □□□ Diff (□□□□□□□□□). □□□□□
□□□□ □□□□□□□□ □□□ □□□□□□□ □□□□ □□□□□□ □□□
□□□□□ □□ □□□□□ □□□ Shadow □□□□□□□□. □□□□□□□□
□□□□□□□□ □□□□□□□□□□ □□: □□□□□□ □□□□□□ □□□□
□□□□□□□□ □□□ □□□□□□ □□□□□□□ □□□□ Shadow □□□□□□□□
□□□□□□□□ □□□ Myers' Diff Algorithm. □□□□□□□ □□□□□ □□□ Diff
□□□ □□□□□□ □□ □□□ □□□□□ □□□ Shadow. □□□□□□□□
□□□□□□□ □□□□ □□□□ Diff □□□ □□□□□ □□□□□□ □□□ Shadow.
□□□□□□□ □□□□□□ □□□□ □□ □□□□□□ □□□□□ □□
□□□□□□□ □□□□ □□□□□□□□ □□□□□□□. □□□□□□□ □□□□□□
□□□□□ □□□□□ □□□□□□□ □□□□ Shadow. □□ □□□□□□ □□□
□□□□□□□□□□□: □□□□□□□ □□□□□□□ □□□□□□ □□ OT□

□□□□□□ □□□ □□□□□□□□ □□ □□□□□□□□
□□□□□□□□□□□□□□□□ □□□□□□□□□□□□ □□ □□□□□□
□□□□□□□□□□. □□□□□□□□□□: □□□□□□□ □□□□□□□ Shadow □□□□
□□□□□□ □□□□□□□□□□□□□□□□ □□ □□□□□□□□□□ □□□□□□□□□□.

4.3 Operational Transformation (OT)

□□□□ Operational Transformation □□ □□□□□ □□□□□□□□□□
□□□□□□□□□ □□□□□□□□□□□□ □□□□□ C. Ellis □ S. Gibbs □□□□ 1989.
□□□□□□□□ □□ □□□□□□□□ □□□□ Google Wave (□□□□□□□□) □□□□□□□□
□□ □□□□□□ □□□□□□□□ □□□□□□□□□□. □□□□□□□ □□□□□□□□ □□ OT
□□ □□□□□□ □□□□□□□□□□ □□□ □□□□□□□□ □□□□□□□□□□□□
□□□□□□□□□□ □□ □□□□□□□□□□ □□□□□□. □□□□□□□ □□□□ □□□□
□□□□□□□ A □□□□□□□ □□□ □□ □□□□□□□ 5□ □□□□□ □□□□□□□ B
□□ □□□ □□□□□□ □□□□□□□ □□□ □□ □□□□□□□ 3□ □□□□ □□□□□□ A
□□□□□□□ □□□□□ □□ □□□□□□□ 6 □□□□ □□□□□□□□ □□□□ □□□□□ B.
□□□□□□ OT □□ □□□□□□□ □□□□□□□□□□: □□□□□ □□□□□□□□□□ (Transform
Function □□ T) □□□□□ □□□□□ □□□□□□ □□□□□□ □□□□□ □□□□
□□□□□□ □□□□□□ □□□□□□□ □□□□□□□□□□ (Control Algorithm) □□□□□ □□□□□
□□□□□□ □□□□□□□□□□ □□□□□□□□□□. □□□□□□□ T(a, b) □□□□□□
□□□□□ □□□□□□ □□ a □□□□□ □□□□□ □□ □□□□□□□□□□ □□□□□□ b.
□□ □□□□□□□ OT: □□□□ □□□□□□□□ □□□□□□□□□□ □□□□□□ □□
□□□□□□ □□□□□□□□□□ □□□□□□□ □□□ □□□□□□□□ □□
□□□□□□□□□□□ □□□□□□□□□□□ □□□□□□□ □□ □□□□□□□ □□ □□□□
□□□□□□□□□□. □□ □□□□□□□□: □□□□□□ □□□□□□□□ □□□□□
□□□□□□□□□ □□□□□□□□□□ □□□□□□□ □□□□□□ □□□□□□ □□□□□□
□□□□□□□□□□ □□□□□□□ □□□□□□□ (Scalability) □□ □□□□□□ □□□□
□□□□□□□□□□□□.

5. CRDTs and Consistent Replication

CRDTs are a class of data structures that allow for consistent replication of data across multiple nodes in a distributed system. They are designed to handle concurrent updates from multiple nodes and ensure that all nodes eventually reach a consistent state. CRDTs are used in a variety of applications, including collaborative editing, distributed databases, and distributed caches.

CRDT Type	Delta-CRDT	Differential Sync	OT
Set	Set	Set	Set
Map	Map	Map	Map
Vector	Vector	Vector	Vector
Off-line	Off-line	Off-line	Off-line
Replicated	Replicated	Replicated	Replicated
Consistent	Consistent	Consistent	Consistent
Examples	Redis, Riak	Google Docs	Google Wave

Figure 2: CRDTs and Consistent Replication

5.1 Delta-CRDTs and Differential Sync

Delta-CRDTs are a type of CRDT that use a delta-based approach to replication. They are designed to handle concurrent updates from multiple nodes and ensure that all nodes eventually reach a consistent state. Delta-CRDTs are used in a variety of applications, including collaborative editing, distributed databases, and distributed caches. Differential Sync is a replication strategy that uses a differential approach to replication. It is designed to handle concurrent updates from multiple nodes and ensure that all nodes eventually reach a consistent state. Differential Sync is used in a variety of applications, including collaborative editing, distributed databases, and distributed caches.

□□□□□□□□ □□□□□□ □□□□□□ □□□□□□.
□□□□□□□□□□□□□□ □□□ □□□□□□ □□□□□□ □□□□□□
□□□□□□□□□□□□□□□□ □□□□□□□□□□. □□□□ **Operational**
Transformation □□□□□:□□□□□ □□□ □□□□ □□□□ □□ □□
□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□ □□□□□□
□□□□□□□□□□□□ □□□□□□ □□□ □□□□□□□□□□□□ □□□□ □□□□.
□□□□□□ □□□□□□□□ □□□ □□□□□□□□□□□□□□ □□□□□□□□□□
□□□□□ □□□□□ □□□□□□□□□□□□□□□□ □□□□□□□□.

6. □□□□□□□□ □□□□□□

□□□□□□ □□□□□□ **Delta** □□ □□□□□□ □□□□□ □□
 □□□□□□□□ □□□□□□□□ □□ □□□□□□□□ □□□□□□□□
 □□□□□□□ □□□ □□□□□□□ □□□□□□□ □□□□□□□. □□□□□□□
 □□ □□□ □□□□□ □□□□ □□□ □□□□□□□□□ □□□□□□ □□□□□□
 □□□□□□□ **Delta** □□□□□

6.1 面向对象编程

1. **Offline-First**: 应用程序首先运行在本地，即使没有网络连接也能使用。当有网络时，数据会自动同步到云端。

- 优点：用户体验好，数据安全可靠。
- 缺点：数据同步可能复杂，需要处理冲突。

2. **Hybrid**: 结合本地和云端存储。部分数据存储在本地，部分存储在云端。

- 优点：灵活性高，可以根据需要调整存储策略。
- 缺点：实现复杂，需要管理本地和云端的数据一致性。

3. **Cloud-First**: 数据主要存储在云端，本地只是缓存。

- 优点：数据集中，易于管理和备份。
- 缺点：网络依赖性强，离线时无法使用。

4. **Edge-First**: 数据存储在边缘设备（如手机、服务器边缘），靠近用户。

- 优点：延迟低，带宽节省。
- 缺点：数据分布广，管理难度大。

5. **Multi-Cloud**: 数据分布在多个不同的云服务提供商上。

- 优点：避免单一供应商锁定，提高容灾能力。
- 缺点：管理复杂，成本可能较高。

6. **Blockchain**: 使用区块链技术进行数据存储，确保数据不可篡改和透明。

- 优点：数据安全性高，透明度高。
- 缺点：性能较低，成本较高。

7. **Serverless**: 使用无服务器架构，数据存储在云端的函数计算服务中。

- 优点：按需付费，扩展性强。
- 缺点：可能不适合大规模数据存储。

8. **Edge Computing**: 在数据产生的边缘进行计算和存储，减少数据传输。

- 优点：降低延迟，节省带宽。
- 缺点：设备资源有限，管理复杂。

9. **Quantum**: 利用量子计算技术进行数据存储和加密。

- 优点：理论上具有极高的安全性和计算能力。
- 缺点：目前处于早期阶段，成本极高。

10. **AI-Driven**: 使用人工智能技术优化数据存储和检索。

- 优点：提高数据利用率和检索效率。
- 缺点：需要大量的数据和算力支持。

6.2 環境問題と経済問題

□□□□ □□□□□□□□ □□□□□□□□ (Collaborative Editing) □□□ □□□
 □□□□□□□□ □□□□□□ Delta□ □□□□ □□□□ □□□□ □□□□□□□□
 □□□□□□□□ □□□□ □□□□□□□□ □□ □□□□□□ □□□□□□□□. □□□□□ □□□□□
 □□□□ □□□□ □□ Google Docs□ □□□□□ □□□□□ □□□□□□□□
 □□□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□ □□□□□□□□. □□□□□□ □□□□

7. 0000 0000000000

000000 000000 Delta 000000 000 000000 000000 00
0000 0000000000 0000 0000 000000 0000000000
0000000000 00000000. 00000000 000 0000000000 00
0000 000000.

7.1 00000 00000 000000000

000000 000000 0000000000 00000000 00 0000000 000
00000 0000000 00000. 0000 00 000000 00 0000 0000000
000000 (UUID) 000000 00 0000000000 000000000000 000
000000000 00 00000000 0000 000000000 0000000000. 00000
0000 000000 00000 0000000000 0000000000: lastModified (0000
0000 000000) 0 version (0000 00000000) 0 origin (00000
000000000) 0 deviceId (00000 00000000). 00 000000 00000000
0000000 00000000 CRDT 00000000 0000 0000. 00000000000
00000000 PN-Counter. 00000000000000 00000000 OR-Set. 000000
00000000000 00000000 LWW-Register. 000000 0000000 0000000000
0000000000 0000 00000000000 0000 000000. 0000 0000000 000000
0000000000 0000000000 00000000000 00000 0000000000 0000
0000000. 0000000000 0000 0000000000 0000000000 00 000000:
0000000 0000000000 0000000000 0000000000 0000000000
00000000000 00000000 0000000000 0000000000.

7.2 00000000000000 00 00000000000

00000 00000000000000 00 000000000000 00 0000 0000000000
00 0000000 00000 000000000000. 000000 0000 00000000000000
0000000: 0000000000 Last Write Wins (LWW): 0000000000 00000000
000000. 00000000 0000 00 000000 0000000000 00000000. 00000000
000000000000 0000 00000000000 0000 00000000000000. 0000000000
Field-Level Merge: 0000 0000 0000000 0000000000 00 0000 00000000
000000 00000000. 000000 0000 0000000 000000 0000000000.
000000000 0000000000 000000000 000000000. 00000000000 Local Wins

8. □□□□□□□□ □□□□□□□□

□□□□□ □□□□□ □□□□□□ **Delta** □□□□□□ □□ □□□□□□□□□

□□□□□□□ □□□□□□□□. □□□□□□□ □□□ □□□□□□□□□ □□

□□□□□□□ □□□□□□□□□□ □□□□ □□□□□

8.1 数据模型与数据库

[illegible]

8.2 数据库的部署与配置

[illegible]

□□□□□□□□: □□□□ □□□□ □□□□□□□□ □□
□□□□□□□□□□□□□□ □□□□□□ □□□□□□□□.

8.3 □□□□□□ □□□□□□□□

□□□ □□□□□□□□ □□□□□□□ □□□ □□□□□□ □□□□□
□□□□□□□□ □□□□□ □□□□□□□. □□□□□□ □□□□: □□□□□□
□□□□□□□: □□□□□ □□□□□□□□ □□□□□ □□□□□ (TLS) □□□□□□
□□□□□□□ (Encryption at Rest). □□□□□□□ □□□□□□□□: □□□□□□
□□ □□□□ □□□□□□ □□□ □□□□ □□ □□□□□□ □□□□□□□□ JWT
□□ OAuth. □□□□□□□ □□□□□□□: □□□□□□ □□ □□□□□□□
□□□□□□□□ □□□□□□ □□□ □□□. □□□□□□□ □□□□□□□: □□□□□□
□□□□□ □□□□ □□□□□□ □□□□□□□□ □□ □□□□□□□
□□□□□□□□ □□□□□□. □□□□□□□ □□□□□□□: □□□ □□□□□
□□□□□□ □□□□□ □□□□□□□ □□□ □□□□□□ □□□. □□□□□□□
□□□□□□□ (Masking): □□□□□ □□□□□□□□ □□□□□□□ □□□
□□□□□ □□□□□□□□ □□□ □□□□□ □□ □□□□□□□.

9. □□□□□□ □□□□□□ Delta

□□□□ □□□□□□ □□□□□□ Delta □□□□□□□□ □□□□□□□□
□□□□□□□□ □□□□□□□□ □□□□□□. □□ □□□□ □□□□□□□□□□
□□□□□□□□: □□□□□□ □□ □□□□□□ □□□□□□□□: □□□□□□□□
□□□□□□ □□□□□□ □□□□□ □□□□□□ □□□□□□□□□□
□□□□□□□□ □□□□□□□□ □□□□ □□□□. □□□□□□ □□□ Delta
□□□□□ □□□ □□□□□ □□□□□□□□. □□□□□□□ □□□ □□□□□
□□□□□□ □□□□□: CRDTs □□□□□□□□ □□□□□□□ □□□□□□□ □□□
□□□□□□ □□□□□□□□ □□□□□□□□ □□□□□□ □□□□□□□□. □□□□
□□□□□□□ □□□□□□□ □□□ □□□□□□□ □□□□□□ □□□□□□
□□□□□. □□□□□□□ □□□□□ □□□□□□□: □□□□□□□□□ Delta
□□□□ □□□□□ □□ □□□□□ □□□. □□□ □□□ Edge Computing
□□□□□□ □□□ □□□□□□□□□□. □□□□□□□ □□□□□□□ □□
□□□□□□ Web3: □□□ □□□□□□□□ □□□□□□□□□□ □□□□ Blockchain.
□□□□□ □□□□□□ □□□□ □□□□ □□□□□. □□□□□□□ □□□□□
□□□□□ □□□□□: □□□□□□ □□□□□ □□□□□ □□□□ □□□
□□□□□□. □□□□□ □□□□□ □□□□□□ □□□□□ □□□□□□. □□□□□□
□□□□□□□ □□□□□□ □□□□□: □□□□□ □□□□□□ □□□□□□□
□□□□□□ Delta □□□ □□□□□□□ □□□□□□□□. □□□□□□□□□□
□□□□□ □□□□□□□□ □□□□□□□□□□.

10. CRDT 的进阶应用

在前面的章节中，我们介绍了 Delta 和 CRDT 的基本概念。在本章中，我们将探讨 CRDT 的进阶应用，包括 Delta-CRDT、Differential Sync、OT、Field-level Delta 以及 Yjs 的 Automerge。

Delta-CRDT 是一种结合了 Delta 和 CRDT 的分布式数据类型。它允许在分布式系统中高效地存储和同步数据。Delta-CRDT 的核心思想是将数据的变化（Delta）与 CRDT 的复制和合并操作结合起来。通过这种方式，Delta-CRDT 可以在分布式系统中实现高效的增量同步。

Differential Sync 是一种基于 Delta 的同步协议。它允许在分布式系统中高效地同步数据。Differential Sync 的核心思想是将数据的变化（Delta）与同步操作结合起来。通过这种方式，Differential Sync 可以在分布式系统中实现高效的增量同步。

OT（Operational Transform）是一种用于解决分布式系统中操作冲突的算法。它允许在分布式系统中高效地同步数据。OT 的核心思想是将操作（Operations）与同步操作结合起来。通过这种方式，OT 可以在分布式系统中实现高效的增量同步。

Field-level Delta 是一种用于解决分布式系统中操作冲突的算法。它允许在分布式系统中高效地同步数据。Field-level Delta 的核心思想是将操作（Operations）与同步操作结合起来。通过这种方式，Field-level Delta 可以在分布式系统中实现高效的增量同步。

Yjs 是一个基于 CRDT 的分布式数据类型库。它提供了多种数据类型，包括文本、列表、字典等。Yjs 的核心思想是将 CRDT 与操作（Operations）结合起来。通过这种方式，Yjs 可以在分布式系统中实现高效的增量同步。

Automerge 是 Yjs 的一个功能，它允许在分布式系统中高效地同步数据。Automerge 的核心思想是将操作（Operations）与同步操作结合起来。通过这种方式，Automerge 可以在分布式系统中实现高效的增量同步。

1. Almeida, P. S., et al. (2015). "Delta-State Replicated Data Types." Journal of Parallel and Distributed Computing.
2. Fraser, N. (2009). "Differential Synchronization." Proceedings of the 9th ACM symposium on Document engineering.
3. Shapiro, M., et al. (2011). "A Comprehensive Study of Convergent and Commutative Replicated Data Types." INRIA Technical Report.
4. Ellis, C. A., & Gibbs, S. J. (1989). "Concurrency control in groupware systems." ACM SIGMOD Record.
5. Kleppmann, M. (2017). "Designing Data-Intensive Applications." O'Reilly Media.
6. IBM Developer. (2024). "Offline data synchronization strategies for mobile apps." IBM Developer Documentation.
7. Google Research. (2009). "Differential Synchronization." Google Research Publications.
8. CouchDB Documentation. (2025). "Replication Protocol." Apache CouchDB Official Documentation.
9. Firebase Documentation. (2025). "Realtime Database Offline Capabilities." Google Firebase Documentation.
10. Yjs Library. (2025). "CRDT Algorithm Implementation." GitHub Repository.